



WEEK 7 · CAPSTONE EXTENSION

การเชื่อมต่อ Chatbot กับ Cloud LLM

ต่อยอดจาก Local LLM สู่การเชื่อมต่อผู้ให้บริการ AI บนคลาวด์ และ deploy ระบบให้ใช้งานได้จริง

หลักสูตร IT · KMITL



ภาพรวมเนื้อหาวันนี้



PART 1

Local สู่ Cloud LLM

- ทวนสถาปัตยกรรม LM Studio เดิม
- ข้อจำกัดของ Local LLM
- Cloud LLM & OpenAI-compatible API



PART 2

เชื่อมต่อจริงในโค้ด

- 3 ตัวแปรหัวใจใน .env
- OpenRouter API protocol
- llm_client.py & data flow



PART 3

Deploy & ข้อจำกัด

- Deploy ขึ้น Vercel
- ปัญหา conversation_store
- แนวทางแก้ไข & การบ้าน



ทวนสถาปัตยกรรมเดิมจาก Week 6

Chatbot เดิมคุยกับโมเดลที่รันอยู่ในเครื่องของเราเองเท่านั้น



Browser

chat.js / fetch



FastAPI

routers/chat.py



LM Studio (Local)

127.0.0.1:1234/v1



ข้อจำกัดของ Local LLM

- 01 ต้องเปิดเครื่องและเปิด LM Studio ทิ้งไว้ตลอด — ถ้าปิดเครื่องหรือปิดโปรแกรม chatbot ก็เชื่อมต่อไม่ได้ทันที
- 02 เอาให้คนอื่นใช้งานผ่านอินเทอร์เน็ตจริงไม่ได้ — LM Studio รันอยู่ที่ `127.0.0.1` ของเครื่องเราเท่านั้น คนนอกเครือข่ายเรียกใช้งานไม่ได้
- 03 จำกัดด้วยสเปกเครื่องของผู้ใช้เอง — โมเดลใหญ่ต้องการ GPU/RAM สูง ซึ่งเครื่องทั่วไปอาจรับไม่ไหวหรือช้ามาก
- 04 ใช้งานร่วมกันหลายคนพร้อมกันยาก — เครื่องคนเดียวรับโหลดจากผู้ใช้งานจริงหลายคนพร้อมกันไม่ได้



Cloud LLM คืออะไร

ผู้ให้บริการรันโมเดลบนเซิร์ฟเวอร์ของเขาเอง เราเรียกใช้งานผ่าน API เท่านั้น

ผู้ให้บริการ	ลักษณะ base URL	หมายเหตุ
OpenRouter	<code>https://openrouter.ai/api/v1</code>	รวมหลายโมเดล/หลายค่ายไว้ endpoint เดียว
OpenAI	<code>https://api.openai.com/v1</code>	เจ้าของฟอร์แมต Chat Completions ดั้งเดิม
Google Gemini	<code>https://generativelanguage.googleapis.com/v1beta/openai</code>	มี OpenAI-compatible endpoint ให้เช่นกัน
Groq	<code>https://api.groq.com/openai/v1</code>	เน้นความเร็วในการตอบสนอง (inference เร็วมาก)

00

มาตรฐาน OpenAI-compatible Chat Completions API

- 01 รูปแบบ request เดียวกันทุกค่าย — ส่ง `{model, messages: [{role, content}]}` เหมือนกันหมด
- 02 รูปแบบ response เดียวกัน — ตอบกลับเป็น `choices[0].message.content` ทุกผู้ให้บริการ
- 03 Endpoint pattern ซ้ำกัน — ทุกเจ้ามี `/chat/completions` และ `/models`
- 04 ผลคือย้ายผู้ให้บริการ = แก้แค่ config — เปลี่ยนแค่ `LLM_BASE_URL`, `LLM_API_KEY`, `LLM_MODEL` ใน `.env` โดยไม่ต้องแตะโค้ด `llm_client.py` เลย



องค์ประกอบ & เครื่องมือที่ใช้ในระบบ

แผนที่รวมทุกชิ้นส่วนก่อนเจาะรายละเอียดทีละส่วน

องค์ประกอบ	หน้าที่	เครื่องมือ/ไลบรารีที่ใช้
หน้าเว็บ (Frontend)	ส่งข้อความ, รับคำตอบแบบสตรีม	<code>chat.js</code> — Fetch API
Web Framework	รับ request, จัดการ route/session	<code>FastAPI</code>
การตั้งค่า	อ่านค่าจาก <code>.env</code> แบบ type-safe	<code>Pydantic Settings</code>
เรียก LLM ภายนอก	ยิง HTTP request แบบ async	<code>httpx</code> (AsyncClient)
ผู้ให้บริการ Cloud LLM	รันโมเดล AI จริง	OpenRouter (OpenAI-compatible API)
การ Deploy	รันโค้ดให้ใช้งานได้นบนอินเทอร์เน็ตจริง	Vercel (Serverless Functions)



ขยายความ: Pydantic Settings คืออะไร

หลักการเบื้องหลังไฟล์ config.py ที่ใช้อ่านค่าจาก .env

- 01 Type-safe configuration** — ระบุชนิดข้อมูลของแต่ละค่าตั้งแต่ตอนประกาศ เช่น `LLM_BASE_URL: str` ผิดชนิดจะ error กันที่ตอนโปรแกรมเริ่มทำงาน ไม่ใช่ตอนใช้งานจริง
- 02 Validate ค่าตั้งแต่แอปเริ่มต้น** — ถ้าลืมตั้งค่าตัวแปรที่จำเป็นใน .env โปรแกรมจะฟ้อง error กันที่ตอนสร้าง `Settings()` ไม่ต้องรอไปเจอ bug ตอนคุยกับ LLM
- 03 ต่างจากการอ่าน `os.environ` ตรงๆ** — `os.environ.get()` คืนค่าเป็น string เสมอและไม่ตรวจสอบว่ามีค่าหรือไม่ ทำให้ bug โผล่ตอนรันจริงแทน
- 04 แหล่งค่าที่ยืดหยุ่น** — อ่านจาก .env ตอนรันในเครื่อง หรือจาก Environment Variables ที่ตั้งบน dashboard ตอน deploy ได้โดยไม่ต้องแก้โค้ด



3 ตัวแปรหัวใจใน .env

- 01 `LLM_BASE_URL` — ที่อยู่ของผู้ให้บริการ ต้องจบด้วย `/v1` เสมอ เช่น `https://openrouter.ai/api/v1`
- 02 `LLM_API_KEY` — กุญแจยืนยันตัวตนที่ได้จาก dashboard ของผู้ให้บริการ ห้าม commit ขึ้น git เด็ดขาด
- 03 `LLM_MODEL` — ชื่อโมเดลที่จะเรียกใช้ เช่น `openai/gpt-4o-mini`
- 04 ตั้งชื่อตัวแปรแบบกลางๆ โดยตั้งใจ — ไม่ผูกกับผู้ให้บริการรายใดรายหนึ่ง เพราะโค้ดคุยด้วยฟอร์แมตเดียวกันทุกเจ้า



สมัครและสร้าง API Key (ตัวอย่าง OpenRouter)

1 สมัครบัญชีที่ openrouter.ai

ล็อกอินด้วย Google/GitHub ได้ทันที ไม่ต้องใส่บัตรเครดิตสำหรับโมเดลฟรี

2 เข้าเมนู Keys ในหน้า dashboard

กด Create Key ตั้งชื่อ key ให้จำง่าย เช่น ตามชื่อโปรเจกต์

3 คัดลอกค่า API key ที่สร้างขึ้น

ระบบจะโชว์ค่าเต็มให้เห็นครั้งเดียว หลังจากนั้นจะเห็นแค่บางส่วนเท่านั้น

4 เลือกโมเดลที่จะใช้จากหน้า Models

มีทั้งโมเดลฟรี (ต่อท้ายด้วย **: free**) และโมเดลเสียเงินตามการใช้งานจริง



OpenRouter API Protocol & Endpoint

ใช้มาตรฐาน OpenAI-compatible ตรงตัว — endpoint และ header เหมือนกันทุกเจ้า

openrouter-api

ขอคุยกับโมเดล (เหมือน OpenAI Chat Completions ทุกประการ)

POST https://openrouter.ai/api/v1/chat/completions

Authorization: Bearer <LLM_API_KEY>

Content-Type: application/json

{ "model": "openai/gpt-4o-mini", "messages": [...] }

ดูรายชื่อโมเดลที่เรียกใช้ได้จริง

GET https://openrouter.ai/api/v1/models

ชื่อโมเดลเป็นรูปแบบ provider/model เสมอ

anthropic/claude-3.5-sonnet · google/gemini-2.0-flash-exp:free



ตั้งค่า .env และหา Model Key ที่ถูกต้อง

อ้างอิงจาก routers/chat.py::GET /api/v1/models ที่ proxy ไปให้ฝั่ง frontend

env + curl

```
# .env ของโปรเจกต์
LLM_BASE_URL=https://openrouter.ai/api/v1
LLM_API_KEY=sk-or-v1-xxxxxxx
LLM_MODEL=openai/gpt-4o-mini

# เช็ค model key ที่ถูกต้องผ่านแอปตัวเอง
> curl http://127.0.0.1:8000/api/v1/models
{"models":[{"id":"openai/gpt-4o-mini", ...}]}
```



ขยายความ: httpx และ Async I/O

ทำไม `services/llm_client.py` ต้องเขียนแบบ `async` ทั้งหมด

- 01 รอคำตอบจาก Cloud LLM ใช้เวลานาน — การเรียก API ภายนอกต้องรอ network round-trip เป็นวินาที ถ้าเขียนแบบ `sync` ธรรมดา โปรเซสจะหยุดรอเฉยๆ รับ request อื่นไม่ได้เลย
- 02 `async/await` ทำให้รอได้โดยไม่บล็อก — ระหว่างรอคำตอบจาก LLM โปรเซสเดียวกันยังไปจัดการ request อื่นที่เข้ามาพร้อมกันได้ต่อ
- 03 `httpx.AsyncClient` รองรับ `async` โดยตรง — ออกแบบมาให้ใช้คู่กับ FastAPI (ซึ่งเป็น `async framework`) ได้อย่างเป็นธรรมชาติ
- 04 กำหนด `timeout` แยกเป็นส่วนๆ — `connect / write / read / pool` แยกกันชัดเจน เพื่อกรณีโมเดลตอบช้าโดยเฉพาะ (ตั้ง `read` ไว้นานถึง 120 วินาที)



ขยายความ: Server-Sent Events (SSE)

โปรโตคอลเบื้องหลังคำตอบที่ค่อยๆ ไหลออกมาทีละตัวอักษร

01 **สตรีมข้อมูลทางเดียวผ่าน HTTP เส้นเดียว** — server ส่งข้อความมาเรื่อยๆ โดยไม่ต้องปิด connection ผ่าน header `Content-Type: text/event-stream`

02 **แต่ละชิ้นข้อมูลขึ้นต้นด้วย `data:`** — ฟังก์ชัน client อ่านทีละบรรทัด ดึงข้อความส่วน `delta` ออกมาต่อกันไปเรื่อยๆ

03 **จบสตรีมด้วย `data: [DONE]`** — เป็น sentinel พิเศษที่ไม่ใช่ JSON บอกว่าโมเดลตอบจบแล้ว

04 **ต่างจาก WebSocket ตรงที่สื่อสารทางเดียว** — SSE เหมาะกับกรณี server พูดย่อยเดียว (เช่น พลัฟร์ AI) เบากว่าและง่ายกว่า WebSocket ที่ต้องคุยได้สองทาง

00

เจาะโค้ด `services/llm_client.py`

01 `_build_messages()` — ประกอบ payload มาตรฐาน — ใส่ system prompt, history ทั้งหมด, แล้วต่อข้อความใหม่ท้ายสุด

02 `chat()` — ยิง `POST /chat/completions` แบบไม่สตรีม รอผลลัพธ์เต็มแล้วคืน `reply_text`

03 `chat_stream()` — ยิงแบบ `stream: true` อ่านทีละบัสรอก SSE แล้ว yield เป็น delta ให้ frontend แบบเรียลไทม์

04 API เป็นแบบ **stateless** เสมอ — ผู้ให้บริการไม่จำบทสนทนาให้ ต้องส่ง history ทั้งหมดไปใหม่ทุกครั้งที่ยิง



จัดการ Error ให้เป็นมิตรกับผู้ใช้

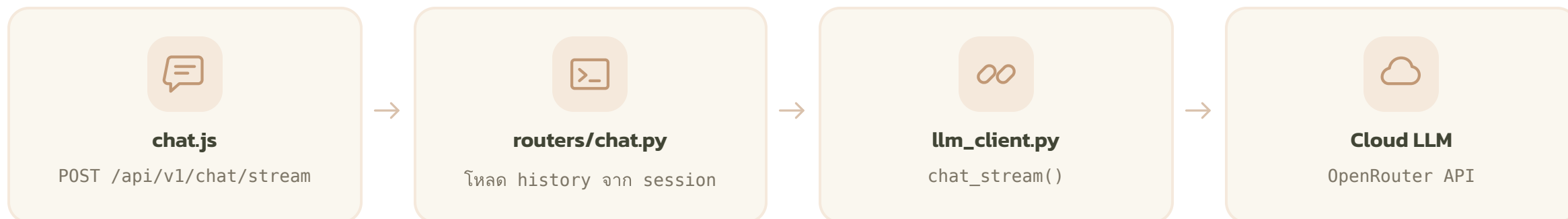
แปล status code ดิบจากผู้ให้บริการ ให้เป็นข้อความไทยที่เข้าใจง่าย

Status	สาเหตุ	ข้อความที่ผู้ใช้เห็น
401	API key ผิดหรือหมดอายุ	ตรวจสอบค่า LLM_API_KEY
404	ไม่พบชื่อโมเดล	ตรวจสอบค่า LLM_MODEL
429	เรียกใช้งานถี่เกินไป	กรุณาลองใหม่ภายหลัง (rate limit)
503	ผู้ให้บริการล่มชั่วคราว	ผู้ให้บริการ AI ไม่พร้อมให้บริการตอนนี้



Data Flow เต็มระบบ

จากการพิมพ์ข้อความ จนถึงคำตอบสตรีนกลับมาแบบเรียลไทม์



← SSE stream ที่: token กลับมาที่ browser แบบเรียลไทม์



บยายความ: Serverless Functions ทำงานอย่างไร

หลักการที่ตั่วอเข้าใจก่อนไป Deploy บั๊น Vercel

- 01 **ไม่มีเซิร์ฟเวอร์ที่รันตลอดเวลา** — โค้ดถูกแพ็ทเป็นฟังก์ชัน แล้วแพลตฟอร์มจะ "ปลุก" บั๊นมารันเฉพาะตอนมี request เข้ามาเท่านั้น
- 02 **Cold start** — ถ้าไม่มีใครเรียกมาสักพัก instance จะถูกปิด ครั้งถัดไปที่มี request ตั่วอเริ่มโปรเซสใหม่ท้งหมด
- 03 **Stateless โดยธรรมชาติ** — แต่ละ request อาจไปตกที่ instance คนละตัว และ instance ไม่การันตีว่าจะอยู่คั่วงให้ request ถัดไปใช้ต่อ
- 04 **เหมาะกับการที่มั่วต้องจำอะไรไว้เอง** — ข้อมูลที่ตั่วออยู่ข้ามหลาย request เช่นประวัติแชท ตั่วอแยกไปเก็บที่อื่นที่ทุก instance เข้าถึงร่วมกันได้เสมอ



Deploy ขึ้น Vercel

- 01 **Push โค้ดขึ้น GitHub repo ของตัวเอง** — Vercel เชื่อมกับ repo แล้วรอ deploy อัตโนมัติทุกครั้งที่มี push
- 02 **New Project → เลือก repo นี้** — Vercel จะอ่าน `vercel.json` แล้วรับ `api/index.py` เป็น serverless function ให้เอง
- 03 **ตั้งค่า Environment Variables** — ใส่ `LLM_BASE_URL` , `LLM_API_KEY` , `LLM_MODEL` , `SYSTEM_PROMPT` ค่าเดียวกับใน `.env`
- 04 **กด Deploy** — ได้ URL จริงที่ใครก็เข้าถึง chatbot ได้ทันที ไม่ต้องเปิดเครื่องตัวเองทิ้งไว้อีกต่อไป



ข้อจำกัดสำคัญ: conversation_store แบบ In-memory

ทำงานได้เฉพาะตอนรันในเครื่องเท่านั้น ไม่ปลอดภัยบน serverless

- 01 เก็บ history ไว้ใน dict ของโปรเซส — `conversation_store: dict[str, list]` อยู่ในหน่วยความจำล้วนๆ
- 02 บน Vercel แต่ละ request อาจไปตกคนละ instance — serverless function ไม่การันตีว่าจะวิ่งไปที่โปรเซสเดิมทุกครั้ง
- 03 Instance ถูกปิดทิ้งได้ตลอดเวลา — เมื่อ instance ถูกรีไซเคิล ข้อมูลในหน่วยความจำก็หายไปทันที
- 04 ผลคือ AI อาจ "ลืม" บทสนทนา — ผู้ใช้คุยต่อเนื่องได้บ้างไม่ได้บ้างแบบไม่สม่ำเสมอ ขึ้นกับว่าตกไป instance ไหน



แนวทางแก้ไข: External Store

- 01 ย้าย **conversation_store** ออกไปเก็บภายนอก — ใช้ที่เก็บกลางที่ทุก instance เข้าถึงร่วมกันได้ แทนหน่วยความจำในโปรเซส
- 02 ตัวเลือดยอดนิยม: **Vercel KV (Redis)** — เขียน/อ่านเร็ว เหมาะกับข้อมูลชั่วคราวอย่างประวัติแชท
- 03 หรือใช้ฐานข้อมูลปกติ — เช่น Postgres/MongoDB ถ้าต้องการเก็บประวัติถาวรระยะยาว
- 04 **Key** ยังใช้ **session_id** เดิมได้เลย — โครงสร้างโค้ดฝั่ง router แทบไม่ต้องเปลี่ยน แค่สลับที่เก็บข้อมูลเบื้องหลัง



สรุปเนื้อหา + แบบฝึกหัด



สรุป

แนวคิด

- Local LLM มีข้อจำกัดเรื่อง deploy
- Cloud LLM ใช้มาตรฐาน OpenAI-compatible



สรุป

การเชื่อมต่อ

- 3 ตัวแปรใน .env
- OpenRouter endpoint & llm_client.py



การบ้าน

ลงมือทำจริง

- สมัคร API key แล้วเชื่อมต่อเอง
- Deploy ขึ้น Vercel และทดสอบใช้งานจริง